Cómo iniciar un proyecto con un LLM — Paso a paso (v1.2.0)

Versión: `1.2.0`
Fecha: 2026-10-18
Archivo: `INICIAR-UN-PROYECTO-PASO-A-PASO-v1.2.0.md`
Propósito: Un procedimiento repetible, comprensible para principiantes, para iniciar un **proyecto de software pequeño o grande** que desarrollarás *junto con* un agente de programación, como Codex, OpenCode o Claude Code.

> **Qué hace esta guía:** Te lleva desde una idea hasta una carpeta de proyecto real, con control de versiones (el **andamiaje inicial**, o *scaffold*), instrucciones útiles, contratos de datos cuando correspondan, habilidades (*skills*) aprobadas y servidores MCP aprobados. **No establece una estructura de carpetas permanente.** El proyecto y su documentación evolucionarán mientras tú y tu agente lo desarrollan.

Antes de comenzar

- **Tú** decides qué quieres conseguir y apruebas las decisiones importantes. **El LLM** ayuda a planificar y redactar archivos. **El agente de programación** trabaja en la carpeta real del proyecto y puede construir y probar el software.
- Un **chat con un LLM** ayuda a redactar planes incluso antes de crear la carpeta. Un **agente de programación** (Codex, OpenCode, Claude Code, etc.) normalmente trabaja con los archivos de esa carpeta.
- **Skill o habilidad:** instrucciones o procedimientos reutilizables para un agente. **Servidor MCP:** conexión que pone herramientas o datos a disposición del agente. No son equivalentes.

- **Scaffold o andamiaje:** directorio, archivos, configuración y pruebas iniciales; no es la aplicación terminada.
- Necesitas una terminal, Git y un agente de programación. Algunas habilidades requieren Node.js/npm (`npx`). Instala solamente las herramientas que necesite la tecnología seleccionada. Los comandos de esta guía utilizan Bash (Linux/macOS/Git Bash/WSL); solicita equivalentes para PowerShell/CMD si los necesitas.
- **Sigue los pasos en orden.** Cuando diga *si corresponde*, anota expresamente **No aplica** y continúa. Un proyecto pequeño no necesita obligatoriamente una habilidad, un servidor MCP, una base de datos, una interfaz gráfica o una biblioteca de validación.
- **Nunca incluyas contraseñas, tokens, credenciales, datos confidenciales de producción o datos personales en prompts, documentos o Git.** Utiliza ejemplos sintéticos o anonimizados.

Elige la configuración más pequeña que sirva para tu proyecto

No crees archivos solo para cumplir con una plantilla. Los pasos se aplican a todos los proyectos, pero la documentación debe ser proporcional al trabajo. Si algo es opcional, escribe **No aplica** en tus notas de planificación y continúa.

Tipo de proyecto	Empieza con	Añade cuando se necesite
Script pequeño o experimento	`README.md`, `AGENTS.md`, programa ejecutable y Git	`PROJECT_SPEC.md` breve; `ARCHITECTURE.md` si existen reglas importantes
Aplicación con interfaz de usuario	Lo anterior, más `DESIGN.md`	Pantallas, reglas de componentes y accesibilidad
Proyecto de datos, importación, API o salida de LLM	Lo anterior, más `DATA_CONTRACTS.md`, `schemas/` u otros contratos pertinentes y pruebas de validación	Migraciones y verificaciones de compatibilidad
Aplicación compleja o colaborativa	Lo anterior, más `ARCHITECTURE.md`, habilidades e integraciones apropiadas	Más pruebas, CI, registros de decisiones y revisiones

Cómo usar esta guía: Sigue los pasos numerados. *Tú* apruebas las decisiones; el *chat de planificación* redacta las especificaciones; tu *agente de programación* crea y modifica los archivos del repositorio real. Puedes adaptar cada prompt. Cada paso indica su resultado esperado. No necesitas preparar una montaña de documentos para un script pequeño.

¿Para qué sirve cada archivo del proyecto?

Archivo o carpeta	Destinatario	Función	Cuándo crear o actualizar
`README.md`	Personas	Qué hace el proyecto y cómo ponerlo en marcha	Andamiaje inicial y cambios para usuarios
`PROJECT_SPEC.md`	Tú y el agente	Alcance, criterios de aceptación y objetivos	Planificación y cambios de requisitos
`AGENTS.md`	Agentes de programación	Comandos, convenciones, restricciones y referencias a otras reglas	Después de crear archivos reales y cuando cambien los procesos
`DESIGN.md`	Agente / participantes de UI	Valores concretos de diseño y reglas de interacción	Cuando existe o cambia una interfaz
`ARCHITECTURE.md`	Agente / responsables técnicos	Reglas invariantes y razones de diseño que no son evidentes	Cuando cambien decisiones arquitectónicas
`DATA_CONTRACTS.md`	Productores, consumidores y agente	Significado y compatibilidad de los datos intercambiados	Cuando cambien las interfaces de datos
`schemas/`	Programas y validadores	Contratos JSON legibles por máquinas, cuando corresponda	Cuando exista o cambie una interfaz JSON estructurada
`skills/`	Agente	Procedimientos reutilizables **locales del proyecto**, cada uno con `SKILL.md`	Cuando se necesite un procedimiento repetitivo particular
`SKILLS.md`	Tú y el agente	Inventario de habilidades seleccionadas, instaladas y globales	Cuando cambien habilidades o versiones
`MCP_SERVERS.md`	Tú y el agente	Inventario, permisos y conexión de MCP, **sin secretos**	Cuando cambien los servidores o su configuración
`scripts/scaffold.sh`	Tú y el agente	Configuración inicial reproducible, no estructura permanente	Creación del andamiaje o cambios deliberados de plantilla

Distinción importante: `README.md` explica el proyecto a las personas; `AGENTS.md` indica cómo debe trabajar el agente *en este repositorio*; `SKILL.md` describe un procedimiento reutilizable en múltiples repositorios. No es necesario copiar una habilidad global a la carpeta `skills/` de cada proyecto; documenta su nombre, origen y forma de uso en `SKILLS.md`. Algunos agentes utilizan otros nombres de archivo o mecanismos de importación: comprueba su funcionamiento real, sin suponer que todos cargan `AGENTS.md` automáticamente.

FASE A — Decide qué vas a construir

PASO 1 — Explica tu idea con palabras sencillas

Escribe de 2 a 5 oraciones. ¿Qué debe hacer? ¿Quién lo utilizará? ¿Cuál es el resultado más importante? No necesitas vocabulario técnico.

Tu acción: Escribe o dicta tu idea aproximada en un chat con un LLM.

Resultado: Una descripción breve e informal.

PASO 2 — Convierte la idea en una especificación pequeña y comprobable

Copia y pega en el LLM:

> Esta es mi idea: [PEGA LA IDEA]. Ayúdame a crear la especificación de la primera versión. Describe el propósito, los usuarios previstos, entre 3 y 6 funciones esenciales, qué queda *fuera del alcance*, cómo comprobaremos que la primera versión funciona, una pila tecnológica recomendada y una estructura inicial sencilla de carpetas. Separa los hechos que proporcioné de tus suposiciones. Pregunta solamente por decisiones que alteren sustancialmente el plan. Guarda la especificación aprobada como `PROJECT_SPEC.md`.

Lee, corrige y aprueba el borrador. Empieza por la **versión útil más pequeña**; podrás añadir funciones después.

Resultado: `PROJECT_SPEC.md` aprobado.

PASO 3 — Aclara requisitos inciertos (entrevista crítica)

Una habilidad de *grilling* formula preguntas enfocadas para descubrir supuestos ocultos. Puedes hacer la entrevista en el chat **sin instalar todavía ninguna habilidad**. Más adelante, `grill-with-docs` puede revisar y documentar las decisiones dentro del repositorio.

Copia y pega:

> Entrevístame sobre el proyecto propuesto. Concéntrate en requisitos ausentes, supuestos contradictorios, usuarios, entradas y salidas, casos extremos, criterios de éxito y restricciones. Haz una sola pregunta importante cada vez. Mantén un registro breve de decisiones y actualiza

> `PROJECT_SPEC.md` después de que apruebe las respuestas. Termina cuando el proyecto esté suficientemente claro para crear el andamiaje.

Resultado: Requisitos más claros y un registro breve de decisiones o preguntas pendientes. **No tienes que resolver todos los detalles futuros de implementación.**

PASO 4 — Redacta los tres documentos de reglas del proyecto

Describen asuntos distintos. **Crea documentos cortos y útiles, no extensos textos especulativos.**

Documento	Pregunta que responde	Cuándo se necesita
`DESIGN.md`	¿Cómo deben verse y comportarse las pantallas?	**Si hay interfaz de usuario** (web, escritorio, móvil o interfaz CLI interactiva significativa)
`ARCHITECTURE.md`	¿Cómo se organiza el sistema y qué reglas nunca deben incumplirse?	**Todos los proyectos**; muy breve para scripts pequeños
`DATA_CONTRACTS.md`	¿Qué datos entran, salen o pasan entre componentes y qué significan?	**Proyectos con intercambio estructurado de datos, archivos, API, configuración o salida estructurada de LLM**

Copia y pega:

> A partir de `PROJECT_SPEC.md`, redacta `ARCHITECTURE.md` (partes del sistema, invariantes importantes y decisiones tecnológicas). Si hay interfaz de usuario, redacta también `DESIGN.md` (valores de diseño, distribución, componentes, estados de carga y error, adaptación a pantallas y accesibilidad). Determina si hay entradas o salidas de datos estructurados; en ese caso, redacta `DATA_CONTRACTS.md` (productores, consumidores, formatos, significado y reglas de validación). Ajusta la extensión al tamaño del proyecto. Si algo no aplica, explica por qué. Señala contradicciones entre los documentos antes de pedirme aprobación.

En un script sencillo sin interfaz, `DESIGN.md` puede marcarse **No aplica** y omitirse. Un script diminuto puede registrar sus pocas reglas técnicas directamente en `AGENTS.md` y prescindir de un archivo `ARCHITECTURE.md` independiente; en sistemas no triviales, mantenlo explícito.

Si creas `DESIGN.md`: Solicita valores concretos, no adjetivos vagos: por ejemplo, metadatos iniciales YAML o una tabla con colores hexadecimales, familias tipográficas, escala de espaciado y radios de esquinas. Incluye distribución, componentes, accesibilidad por teclado, estados de carga/error y comportamiento responsivo. Así evitas que una sesión del agente cambie involuntariamente el diseño de otra. Para una UI pequeña, sé breve.

Resultado: Documentos de reglas adecuados y aprobados.

PASO 5 — Define contratos de datos verificables por máquinas (cuando corresponda)

Si el proyecto recibe o produce datos estructurados, determina **su forma exacta antes de programar los módulos que los consumirán**. Ejemplos: importaciones CSV, solicitudes y respuestas API, configuración JSON, mensajes entre módulos y respuestas estructuradas de LLM.

Un **JSON Schema** describe restricciones sobre datos JSON: nombres, tipos, campos obligatorios, valores permitidos, límites numéricos y estructuras condicionales. **Por sí solo no demuestra todas las reglas del negocio** (por ejemplo, una fórmula actuarial, la unicidad entre registros o la igualdad de dos totales almacenados por separado). Utiliza código y pruebas adicionales para esas reglas. Para CSV, datos binarios, tablas de bases de datos y otros formatos, documenta también el contrato nativo correspondiente: JSON Schema no siempre es lo adecuado.

Copia y pega:

> Examina `PROJECT_SPEC.md`, `ARCHITECTURE.md` y `DATA_CONTRACTS.md` e inventaría cada productor, consumidor, entrada, salida y transformación de datos. Para interfaces JSON, crea esquemas en `schemas/`, indicando expresamente la versión de JSON Schema (prefiere Draft 2020-12 salvo incompatibilidad con las

> herramientas). Define campos obligatorios, tipos, enumeraciones, valores nulos, unidades, fechas, identificadores, rangos, campos opcionales y tratamiento de errores. Para formatos distintos de JSON, define un contrato equivalente. Genera ejemplos **sintéticos** válidos, inválidos y extremos e identifica reglas que requieren pruebas programadas adicionales. No inventes nombres de campos ni significados técnicos: enumera las dudas. Explica cómo validar y versionar los contratos.

Estructura pequeña sugerida (solo lo necesario):

```
schemas/
  inputs/
  outputs/
  config/
  examples/valid/
  examples/invalid/
tests/contracts/
```

Crea únicamente las subcarpetas utilizadas. Una API, por ejemplo, podría tener `schemas/api/create-order.schema.json`, y una configuración de agente `schemas/config/opencode.schema.json`. No fuerces JSON Schema sobre CSV ni sobre datos relacionales: utiliza contratos propios del formato. Define una política de compatibilidad: los cambios que rompan un productor o consumidor existente requieren revisión, migración o publicación coordinada. Determina si la versión del contrato figurará en el nombre, el identificador del esquema, la versión API u otro mecanismo explícito.

Resultado: `DATA_CONTRACTS.md` aprobado, esquemas pertinentes (por ejemplo, `schemas/inputs/import.schema.json`), ejemplos sintéticos y plan de validación. Si no hay intercambio estructurado, anota **No aplica**.

FASE B — Selecciona habilidades del agente y conexiones MCP

Distinción importante: En esta fase **seleccionas y revisas** las herramientas. La instalación local se realizará **después de crear la carpeta y el repositorio Git**, para evitar problemas de orden.

PASO 6 — Examina un conjunto básico de habilidades reutilizables

Estas son **candidatas para considerar en casi cualquier proyecto**, no una orden de instalarlas todas automáticamente. Antes de instalar, comprueba sus nombres y orígenes actuales: los repositorios pueden cambiar.

Habilidad	Uso adecuado
`grill-me` **o** `grill-with-docs`	Cuestionar requisitos. Utiliza **una modalidad principal de entrevista**; `grill-with-docs` es especialmente útil cuando ya existen archivos y deben documentarse decisiones.
`find-skills`	Buscar habilidades cuando surja una necesidad. Descubrir una habilidad **no autoriza** instalarla automáticamente.
`to-prd` → `to-issues`	Convertir requisitos aprobados en un documento de requisitos de producto (PRD), y luego en tareas. Ejecuta `to-issues` **solo después** de aprobar el PRD y preparar un gestor de incidencias.
`improve-codebase-architecture`	Revisar o refactorizar código **ya existente**. **No** lo ejecutes obligatoriamente sobre un andamiaje vacío.
`setup-matt-pocock-skills`	Configurar el flujo auxiliar en un repositorio apropiado, ya inicializado, si utilizas ese conjunto.

Para consultar la oferta real (sin instalar a ciegas):

```
npx skills@latest add mattpocock/skills --list
npx skills@latest add vercel-labs/skills --list
```

Comandos de instalación específicos, como ejemplo (ejecuta después del Paso 12, dentro del proyecto, solo si están aprobados y son compatibles con tu agente):

```
npx skills@latest add mattpocock/skills --skill grill-with-docs
npx skills@latest add vercel-labs/skills --skill find-skills
npx skills@latest add mattpocock/skills --skill to-prd --skill to-issues
npx skills@latest add mattpocock/skills --skill improve-codebase-architecture
npx skills@latest add mattpocock/skills --skill setup-matt-pocock-skills
```

Alternativa opcional a `grill-with-docs`:

```
npx skills@latest add mattpocock/skills --skill grill-me
```

El nombre `prd-to-issues` puede describir el proceso, pero las **dos habilidades indicadas** son `to-prd` y `to-issues`. Según el agente, las habilidades pueden ejecutarse con comandos de barra (como `/to-`

`prd`), por nombre u otra interfaz. **Pregunta a tu agente cómo se invocan; no supongas que todos utilizan los mismos comandos.**

Referencias: https://skills.sh/mattpocock/skills · https://skills.sh/vercel-labs/skills/find-skills

PASO 7 — Añade TUS habilidades obligatorias y solicita recomendaciones opcionales

Copia y pega:

> Las habilidades que exijo son: [NOMBRES, REPOSITORIOS O ENLACES]. Mantén mis elecciones separadas de las recomendaciones opcionales. Comprueba que sean compatibles con mi agente de programación. Recomienda únicamente habilidades adicionales que aporten funciones distintas. Explica cuándo usar cada una; todavía no instales nada.

Dónde se guardan: Una habilidad puede instalarse globalmente para el agente (compartida entre repositorios) o localmente en un proyecto. No copies habilidades globales a cada proyecto salvo que el agente lo exija o necesites fijar deliberadamente una versión local. Cada habilidad local normalmente incluye su `SKILL.md`; por ejemplo `skills/release-checklist/SKILL.md`. Mantén en `SKILLS.md` un inventario de origen, versión y ámbito de instalación.

Resultado: Una propuesta única de inventario: **exigida por el usuario**, **recomendada**, **aceptada/rechazada**, **motivo**, **origen de instalación** y **cuándo invocarla**. Será `SKILLS.md`.

PASO 8 — Comprueba los conflictos entre habilidades antes de instalarlas

Copia y pega:

> Examina la documentación y configuración de cada habilidad seleccionada. Detecta duplicados, instrucciones contradictorias, funciones solapadas, incompatibilidades de

> dependencias o versiones, permisos y archivos de configuración en conflicto. Redacta `SKILL_CONFLICT_REPORT.md` con columnas: Habilidad A, Habilidad B, evidencia, gravedad (Crítica/Moderada/Leve), resolución recomendada y estado. Conserva mis habilidades obligatorias siempre que sea técnicamente posible y seguro; consúltame antes de sustituirlas. Distingue conflictos **verificados** de conflictos **posibles**. Si no encuentras ninguno, informa «No se detectaron conflictos entre habilidades» e indica lo que comprobaste. No instales habilidades con conflictos críticos pendientes.

Resultado: `SKILL_CONFLICT_REPORT.md` aceptado. Los solapamientos compatibles pueden conservarse si se delimitan sus responsabilidades.

PASO 9 — Enumera TUS servidores MCP y considera recomendaciones opcionales

Un servidor MCP proporciona al agente funciones como operaciones con repositorios, bases de datos, servicios externos o documentación. **MCP es opcional:** un script local sencillo quizá no necesite ninguno.

Copia y pega:

> Mis servidores MCP obligatorios son: [LISTA, O «ninguno»]. Indica a qué se conecta cada uno, qué operaciones espero y si el agente seleccionado lo admite. Sugiere únicamente servidores complementarios e incluye sus fuentes oficiales y permisos necesarios. Todavía no los instales ni autorices.

Resultado: Plan `MCP_SERVERS.md` aprobado, o **No se necesitan servidores MCP**.

PASO 10 — Revisa conflictos y permisos de los servidores MCP

Copia y pega:

> Revisa todos los servidores MCP seleccionados para detectar nombres de herramientas duplicados, escrituras simultáneas en conflicto, entornos de ejecución, transportes, versiones, puertos o autenticación incompatibles, accesos a archivos o redes y permisos excesivos. Redacta `MCP_CONFLICT_REPORT.md` con evidencia, gravedad, resolución y estado de verificación. No habilites conflictos críticos sin resolver. No solicites secretos en el chat ni guardes credenciales en archivos del repositorio.

Resultado: `MCP_CONFLICT_REPORT.md` aprobado o una entrada breve **No aplica** si no hay servidores. Revisar configuraciones no demuestra que la conexión funcione.

FASE C — Construye la carpeta inicial del proyecto

PASO 11 — Solicita el script de andamiaje y los archivos iniciales

El script debe **crear archivos reales** y poder repetirse sin destruir silenciosamente tus cambios. **No** debe ejecutar automáticamente instalaciones externas no revisadas, conectar cuentas ni incluir secretos. Los comandos de instalación deben figurar en una lista explícita o en otro script de configuración revisado.

Copia y pega:

> Genera `scripts/scaffold.sh` para mi proyecto aprobado y la tecnología elegida. Debe crear un proyecto inicial mínimo y funcional y escribir el contenido real aprobado en `PROJECT_SPEC.md`, `ARCHITECTURE.md`, `DESIGN.md` (si aplica), `DATA_CONTRACTS.md` y `schemas/` (si aplica), así como `README.md`, `SKILLS.md`, `MCP_SERVERS.md`, `SKILL_CONFLICT_REPORT.md` y `MCP_CONFLICT_REPORT.md`. Añade un `.gitignore` adecuado, una prueba básica de funcionamiento y `tests/contracts/` si existen contratos. No crees carpetas innecesarias. No sobrescribas archivos modificados al repetir la ejecución. Incluye instrucciones para instalar requisitos previos y comandos exactos para ejecutar y comprobar la aplicación inicial.

> Cuando haga falta, coloca las instrucciones revisadas de habilidades y MCP locales en un script independiente `scripts/setup-agent-tools.sh`; nunca guardes secretos. Explica las diferencias entre agentes. Entrega un script completo descargable y los archivos necesarios, no plantillas vacías presentadas como resultados terminados.

El árbol podría ser el siguiente; **tu proyecto puede ser más sencillo o más grande**. No generes `AGENTS.md` mecánicamente antes de tener archivos reales. Documenta las habilidades globales en `SKILLS.md`; crea `skills/` solo si mantienes habilidades locales:

```
my-project/
├── README.md
├── PROJECT_SPEC.md
├── ARCHITECTURE.md
├── DESIGN.md                   # solo proyectos con UI
├── DATA_CONTRACTS.md           # solo proyectos con datos estructurados
├── AGENTS.md                   # creado/revisado con el agente después
├── SKILLS.md
├── MCP_SERVERS.md
├── SKILL_CONFLICT_REPORT.md
├── MCP_CONFLICT_REPORT.md
├── schemas/                    # solo cuando corresponda
├── skills/                     # solo habilidades locales, si existen
│   └── release-checklist/
│       └── SKILL.md
├── tests/                      # pruebas básicas y de contratos
├── scripts/
│   ├── scaffold.sh
│   └── setup-agent-tools.sh    # solo cuando corresponda
└── src/                        # o el punto de entrada adecuado
```

Resultado: Script descargable, contenido inicial revisado e instrucciones claras para ejecutarlo.

PASO 12 — Crea la carpeta, ejecuta el script y comprueba el resultado

Guarda `scaffold.sh` en una ubicación accesible, crea una carpeta y **ejecuta el script desde ella**. Un ejemplo en Bash:

```bash
mkdir my-project
cd my-project
mkdir -p scripts
# Copia el archivo scaffold.sh descargado a scripts/ aquí.
bash scripts/scaffold.sh
```

Si el script generado requiere otra ruta o argumentos, sigue **sus propias** instrucciones. Después, enumera los archivos y ejecuta la prueba básica que se haya generado. No continúes si faltan archivos necesarios o no existe un punto de partida ejecutable.

Resultado: Un directorio real con archivos iniciales funcionales y documentación aprobada.

PASO 13 — Inicializa Git y guarda la primera versión

Desde la raíz del proyecto:

```
git init
# Comprueba .gitignore y asegúrate de no añadir tokens ni datos privados.
git status
git add .
git commit -m "chore: initial project scaffold"
```

Si Git solicita tu identidad, configúrala. Crear un repositorio **remoto** en GitHub y publicar es un paso separado y opcional; el agente puede ayudarte después. Un commit local ya es un punto de recuperación.

Resultado: Primera versión con control de cambios. **Guarda `scripts/scaffold.sh` en Git** para saber cómo se creó la carpeta. El script es una receta histórica de configuración, no una instrucción para reconstruir o sobrescribir el árbol que irá evolucionando. Si lo amplías, hazlo de forma deliberada y segura.

PASO 14 — Instala las habilidades aprobadas y configura los MCP aprobados

Ahora que existe el directorio, utiliza los comandos **revisados** de los Pasos 6–10. Instala herramientas locales desde la raíz del proyecto. Si utilizas `setup-matt-pocock-skills`, ejecútalo **después de inicializar Git**, según los requisitos actuales de esa herramienta. Las instalaciones globales pueden haberse realizado antes.

Copia y pega a tu agente de programación:

> Sigue `SKILLS.md`, `MCP_SERVERS.md` y los dos informes de conflictos. Muéstrame los comandos de instalación aprobados y verifica sus fuentes antes de ejecutarlos. Instala solamente habilidades locales aprobadas, configura solo MCP aprobados

> y usa variables de entorno o un almacén seguro para las credenciales. Comprueba que cada habilidad pueda encontrarse y que cada servidor MCP realmente se conecte y ofrezca las herramientas previstas. Registra éxitos, fallos y versiones o revisiones exactas cuando estén disponibles. No declares que algo funciona si no se ha probado.

Resultado: Herramientas del agente verificadas o una lista explícita de las que faltan y sus impedimentos. **Algunos servidores MCP exigirán que autorices tus cuentas personalmente.**

PASO 15 — Abre el agente en el proyecto e inicializa sus instrucciones

Inicia Codex, OpenCode, Claude Code u otro agente **con la carpeta del proyecto como espacio de trabajo**. Utiliza su función de inicialización, si existe. Algunos admiten `/init`; otros usan comandos o archivos distintos. **Pregúntale al agente; no asumas que `/init` es universal.**

Copia y pega:

> Examina los archivos que realmente existen en este repositorio. Crea o actualiza el archivo de instrucciones del agente que utilice esta herramienta. Prefiere un `AGENTS.md` breve como referencia compartida donde se admita; crea los archivos o directivas de importación que necesite este agente específico. No supongas que todas las herramientas leen `AGENTS.md` automáticamente. Referencia `PROJECT_SPEC.md`, `ARCHITECTURE.md`, `DESIGN.md` (si existe), `DATA_CONTRACTS.md` y `schemas/` (si existen), `SKILLS.md`, `MCP_SERVERS.md` y ambos informes de conflictos. Explica los comandos reales de compilación, ejecución, pruebas y validación; indica cuándo utilizar cada habilidad o MCP aprobado. No copies instrucciones extensas de habilidades a `AGENTS.md`, no inventes archivos ausentes ni incluyas credenciales. Conserva las instrucciones importantes que haya escrito el usuario.

Resultado: Un agente que conoce las reglas del repositorio y sabe dónde encontrar los documentos. Conserva `README.md` para personas y pon las instrucciones operativas en `AGENTS.md`, en lugar de duplicarlas. **Inicializar un agente no demuestra que sus herramientas funcionen: verifica por separado el descubrimiento y las conexiones.**

PASO 16 — Prueba la versión inicial y confirma la configuración del agente en Git

Solicita al agente que ejecute la prueba básica de la aplicación, la validación de esquemas (cuando corresponda) y las pruebas unitarias existentes. Comprueba las referencias entre documentos. Después:

```
git status
git add .
git commit -m "chore: configure agent context and contracts"
```

Si nada cambió desde el último commit, Git indicará correctamente que no hay cambios para registrar.

Resultado: Un punto de partida reproducible y probado para desarrollar con el agente.

FASE D — Desarrolla y conserva la salud del proyecto

PASO 17 — Planifica la primera tarea pequeña de desarrollo

El andamiaje **no** es una aplicación completa. Solicita que el agente convierta la especificación aprobada en cambios pequeños con pruebas. Si escogiste el flujo de Matt Pocock, usa `to-prd` para preparar el PRD y `to-issues` **después** de aprobarlo y configurar el gestor de incidencias. Para un script pequeño, basta una lista sencilla de tareas; no hace falta un gestor formal.

Copia y pega:

> Lee las reglas del repositorio. Propón la función útil más pequeña que debemos implementar, con criterios de aceptación y una forma de probarla. Identifica qué archivos y contratos de datos afecta. Implementa un cambio revisable

> cada vez, ejecuta las pruebas pertinentes, resume los cambios y crea un commit solo después de que apruebe el resultado o te autorice explícitamente a hacerlo.

Resultado: Un ciclo incremental de desarrollo, no una generación gigantesca imposible de verificar.

PASO 18 — Permite que evolucione el árbol del proyecto, pero mantén sus contratos e instrucciones

Durante el desarrollo, el agente puede añadir carpetas, cambiar nombres de módulos, modificar dependencias o proponer otra arquitectura. **Es normal.** `scripts/scaffold.sh` documenta la **configuración inicial**, no obliga a restaurar continuamente el árbol original. Mantén `README.md`, `ARCHITECTURE.md`, `DESIGN.md` y `DATA_CONTRACTS.md` acordes con el software real.

Antes de aceptar un cambio, pregunta:

> ¿Este cambio afecta la arquitectura, la interfaz de usuario, los productores o consumidores de datos, los esquemas, las habilidades, los servidores MCP, los permisos o los comandos de compilación? Si es así, actualiza los documentos y pruebas correspondientes dentro del mismo cambio. Comprueba compatibilidad y migraciones antes de modificar interfaces de datos. Repite la revisión de conflictos si cambian habilidades o servidores MCP. Muéstrame los resultados de verificación y riesgos pendientes.

Repite la inicialización del agente o actualiza `AGENTS.md` cuando cambie significativamente la estructura; **revisa los resultados porque una inicialización automática puede sobrescribir instrucciones personalizadas**. Utiliza `improve-codebase-architecture` solamente cuando exista código sustancial y quieras evaluar su arquitectura. Registra puntos de recuperación en Git y conserva el historial.

Resultado: Un proyecto en crecimiento que tú y las futuras sesiones del agente puedan comprender.

Apéndice — prompts para mantener el contexto del agente

Al crear o actualizar `AGENTS.md`, una instrucción breve puede ser:

> Lee `PROJECT_SPEC.md` para conocer el alcance; `ARCHITECTURE.md` para las invariantes; `DESIGN.md` para normas de UI (si existe); `DATA_CONTRACTS.md` y `schemas/` para interfaces de datos (si existen); y `SKILLS.md` / `MCP_SERVERS.md` para las herramientas aprobadas. Consulta `scripts/scaffold.sh` solo para investigar o modificar la configuración inicial. Utiliza los comandos de compilación y prueba reales de este repositorio; nunca inventes archivos ausentes ni guardes secretos en Git.

Una habilidad local sencilla puede residir en `skills/<skill-name>/SKILL.md` e incluir nombre, descripción, cuándo usarla, procedimiento ordenado y comprobaciones. El descubrimiento automático de esa carpeta **depende de la herramienta**: confirma la ruta exigida en su documentación. Instala fuentes externas solo después de revisar su contenido y permisos.

Cuando cambies el árbol, **actualiza sus referencias**, pero no sobrescribas rutinariamente documentos mantenidos a mano al repetir la inicialización del agente. Revisa las diferencias y registra los cambios en Git.

Lista rápida de comprobación

- [ ] Los objetivos y límites de la primera versión están en `PROJECT_SPEC.md`.
- [ ] `ARCHITECTURE.md` contiene reglas reales; `DESIGN.md` existe si hace falta una UI.
- [ ] Identifiqué productores y consumidores de datos; hay esquemas y pruebas sintéticas cuando corresponde.
- [ ]

Distinguí mis habilidades y servidores MCP obligatorios de las recomendaciones opcionales.

- [ ] Revisé conflictos de habilidades y MCP; resolví los críticos antes de instalar.
- [ ] El andamiaje crea archivos reales y un proyecto inicial ejecutable.
- [ ] Git conserva al menos un commit inicial; no se han registrado secretos.
- [ ] El agente seleccionado puede leer las reglas y usar las herramientas aprobadas.
- [ ] Las pruebas iniciales pasan o están documentados los fallos y bloqueos.
- [ ] Comprendo que el árbol inicial es un punto de partida, no una estructura permanente.

Todo el proceso en una línea

Idea → especificación → aclarar requisitos → diseño/arquitectura/contratos de datos → seleccionar y revisar habilidades/MCP → andamiaje → primera versión en Git → configurar herramientas → inicializar el agente → verificar → desarrollar funciones pequeñas → actualizar documentos, esquemas, pruebas y Git conforme evoluciona el proyecto.

Control de versiones de ESTA norma reutilizable

Esta guía es una **norma viva independiente**, no el número de versión de los proyectos creados con ella.

- Utiliza `MAJOR.MINOR.PATCH`: **MAJOR** para cambios importantes incompatibles en el proceso; **MINOR** para nuevas fases o instrucciones compatibles; **PATCH** para correcciones editoriales.
- Cada publicación modifica la versión del **título**, **nombre de archivo** y **metadatos**, y añade una entrada al historial. Conserva

las versiones anteriores en Git o en un archivo versionado. No sobrescribas una versión antigua con contenido nuevo.

- Commit recomendado: `docs(standard): release v1.2.0`; etiqueta opcional: `project-standard-v1.2.0`.
- Puedes utilizar esta norma con varios sistemas operativos, agentes y lenguajes de programación. Las instrucciones específicas de los agentes pueden cambiar; verifica su documentación oficial antes de ejecutarlas.

Historial de revisiones

Versión	Fecha	Cambios
1.2.0	2026-10-18	Se fusionaron conceptos de Project Bootstrap Kit en una guía única para principiantes: matriz de responsabilidades, habilidades globales frente a locales, valores concretos de diseño, configuraciones mínimas, portabilidad entre agentes, precauciones para repetir el andamiaje y separación entre documentación para personas y agentes.
1.1.0	2026-10-18	Se reorganizó como proceso literal para principiantes; se añadieron `PROJECT_SPEC.md`, condiciones claras para `DESIGN.md`, `DATA_CONTRACTS.md`, diseño y validación de JSON Schema, secuencia de instalación, comprobación del proyecto real, inicialización del agente y mantenimiento durante la evolución del proyecto.
1.0.0	2026-10-18	Primera versión formal: habilidades reutilizables, selección e instalación de MCP, revisiones de conflictos, andamiaje, contexto del agente y control de versiones con Git.